Day 170 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 3 reference

Watch: Choosing the Learning Rate with LR Finder (Apache MXNet) | Competition Winning Learning Rates (Leslie Smith, MLconf)

`day170_learning_rate_finder.py`, segment by segment, in the order `main()` uses things. Today's new parts are lr_range, lr_find, suggest, SGD, bce_logits, train_mlp and mlp_stepper, copied unchanged into day170_exercise.py; Day 126's network and Adam and Day 161's Adult data are imported. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: lr = learning rate · lrs = the finder's rising rates · raw, sm = the losses as measured and smoothed · beta = the smoothing factor · net = a list of (W, b) layers · A = each layer's activations · crit = 2 / (largest eigenvalue) · D = the data: Xa, ya fit; Xv, yv score

Segment 0 — Setup

Imports and constants

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
sys.path.insert(0, os.path.join(ROOT, "week-19-unsupervised-learning-ii"))
sys.path.insert(0, os.path.join(ROOT, "week-24-model-evaluation-statistics"))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day161_exercise as d161
HIDDEN = (64, 32)
EPOCHS = 5
BATCH = 128
FIND_STEPS = 300
LO, HI = 1e-6, 10.0
SEEDS = 5
GRID = 10.0 ** np.arange(-5, 1.01, 0.5)
SWEEP_SEEDS = 3
```

- `sys.path.insert(0, os.path.join(ROOT, ...))` — two earlier weeks' folders go first on the import path, so their exercises import by name.
- `with contextlib.redirect_stdout(io.StringIO()):` — the imported exercises print hand checks; this sends that text to a string nobody reads.
- `LO, HI = 1e-6, 10.0` — tuple unpacking: two names set on one line.
- `10.0 ** np.arange(-5, 1.01, 0.5)` — 13 powers of ten half a decade apart; 1.01, not 1, so the end point 10 is included.

Segment 1 — The parts

lr_range and lr_find

MATH

$lr_t = lo\,(hi/lo)^{t/(T-1)} \quad a_t = \beta a_{t-1} + (1-\beta)loss_t \quad s_t = a_t / (1-\beta^{t+1})$

CODE

```
def lr_range(lo, hi, steps):
    return lo * (hi / lo) ** (np.arange(steps) / (steps - 1))
def lr_find(step, lrs, beta=0.98, blowup=4.0):
    raw, sm, avg, best = [], [], 0.0, np.inf
    for t, lr in enumerate(lrs):
        loss = step(lr); avg = beta * avg + (1 - beta) * loss
        s = avg / (1 - beta ** (t + 1))
        raw.append(loss); sm.append(s)
        if not np.isfinite(s) or s > blowup * best:
            break
        best = min(best, s)
    return np.asarray(lrs[:len(sm)]), np.array(raw), np.array(sm)
```

- `(hi / lo) ** (np.arange(steps) / (steps - 1))` — exponents 0 to 1 in equal steps: the rate grows by the same factor each step.
- `raw, sm, avg, best = [], [], 0.0, np.inf` — four names at once; best starts at infinity so any loss beats it.
- `for t, lr in enumerate(lrs):` — t counts the steps from 0, needed for the bias correction.
- `avg / (1 - beta ** (t + 1))` — the average starts at 0, so early values are too small; this divides that out.
- `if not np.isfinite(s) or s > blowup * best:` — stop on inf or nan, or once the smoothed loss passes four times its best.
- `break` — leaves the loop early; the step that blew up is already recorded.
- `lrs[:len(sm)]` — only the rates actually used.

suggest: two ways to read the curve

MATH

$min / 10 = lr_{argmin\ s} / 10 \quad steepest = lr\ at\ argmin\ over\ t\ of\ ds / d(log\ lr)$

CODE

```
def suggest(lrs, sm, skip=10):
    g = np.gradient(sm, np.log(lrs))
    steep = lrs[skip + int(np.argmin(g[skip:-5]))] if len(sm) > skip + 6 else lrs[0]
    return dict(min_over_10=float(lrs[int(np.argmin(sm))] / 10), steepest=float(steep))
```

- `np.gradient(sm, np.log(lrs))` — the slope at every point, against log rate (central differences inside, one-sided at the ends).
- `g[skip:-5]` — ignores the first 10 points (the average is still settling) and the last 5 (the blow-up).
- `skip + int(np.argmin(...))` — argmin counts from the start of the slice, so skip is added back.
- `... if len(sm) > skip + 6 else lrs[0]` — a run too short to read falls back to its first rate.
- `dict(min_over_10=..., steepest=...)` — keyword arguments make a dictionary with those keys.

SGD and bce_logits

MATH

$$W \leftarrow W - \eta\, \partial loss/\partial W \quad loss = mean[log(1 + e^{z}) - yz] \quad \partial loss/\partial z = (\sigma(z) - y) / n$$

CODE

```
class SGD:
    def __init__(self, net, lr=1e-2):
        self.lr = lr
    def step(self, net, grads):
        for (W, b), (gW, gb) in zip(net, grads):
            W -= self.lr * gW; b -= self.lr * gb
def bce_logits(L, y):
    z = L[:, 0]
    loss = float(np.mean(np.logaddexp(0, z) - y * z))
    return loss, ((d126.sigmoid(z) - y) / len(y))[:, None].astype(np.float32)
```

- `class SGD:` — a class with the same two methods as Day 126's Adam, so either can be passed as opt_cls.
- `self.lr = lr` — stored on the object; the finder changes opt.lr before every step.
- `for (W, b), (gW, gb) in zip(net, grads):` — unpacks each layer's weights and its gradients in one line.
- `W -= self.lr * gW` — in place: W is the array inside net, so the network itself changes; W = W - ... would not.
- `np.logaddexp(0, z)` — $\log(e^{0} + e^{z}) = \log(1 + e^{z})$, computed without overflow for large z.
- `[:, None].astype(np.float32)` — back to one column, and to the network's float32.

train_mlp: one real training run

CODE

```
def train_mlp(Xa, ya, Xv, yv, opt_cls, lr, seed, epochs=EPOCHS, batch=BATCH):
    sizes = [Xa.shape[1], *HIDDEN, 1]; acts = ["relu"] * len(HIDDEN) + ["linear"]
    net = d126.init_net(sizes, seed); opt = opt_cls(net, lr); rng = np.random.RandomState(seed +
        1)
    with np.errstate(all="ignore"):
        for _ in range(epochs):
            o = rng.permutation(len(Xa))
            for s in range(0, len(Xa), batch):
                A = d126.forward(net, Xa[o[s:s + batch]], acts)
                opt.step(net, d126.backward(net, A, acts, bce_logits(A[-1], ya[o[s:s +
                    batch]])[1])[0])
        v = bce_logits(d126.forward(net, Xv, acts)[-1], yv)[0]
    return v if np.isfinite(v) else np.inf
```

- `[Xa.shape[1], *HIDDEN, 1]` — the star unpacks the tuple into the list: [87, 64, 32, 1].
- `["relu"] * len(HIDDEN) + ["linear"]` — list repetition, then one more item: a linear output, because the loss takes logits.
- `opt = opt_cls(net, lr)` — the class itself is an argument; calling it builds the optimiser.
- `RandomState(seed + 1)` — the batch order's own generator, separate from the weights' seed.
- `Xa[o[s:s + batch]]` — the next batch of shuffled rows; the last batch is smaller.
- `d126.backward(...)[0]` — backward returns (grads, dout); [0] keeps the gradients.
- `v if np.isfinite(v) else np.inf` — a diverged run (nan or inf) is scored as infinity, so it never wins.

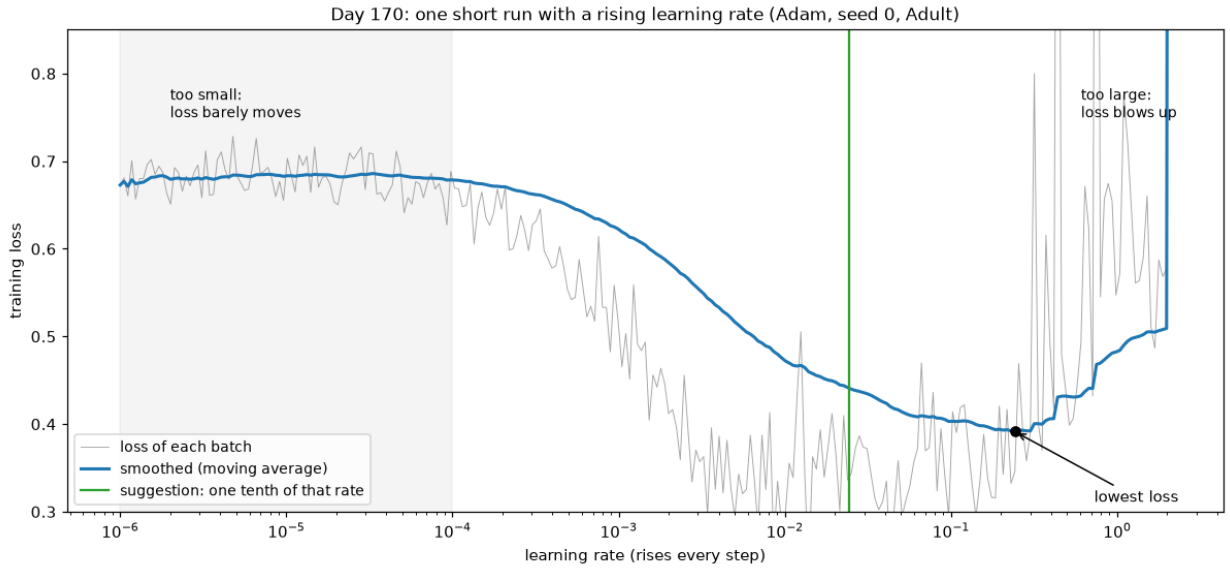
mlp_stepper: a closure for the finder

CODE

```
def mlp_stepper(Xa, ya, opt_cls, seed, batch=BATCH):
    sizes = [Xa.shape[1], *HIDDEN, 1]; acts = ["relu"] * len(HIDDEN) + ["linear"]
    net = d126.init_net(sizes, seed); opt = opt_cls(net, LO); rng = np.random.RandomState(seed +
        1)
    state = dict(o=rng.permutation(len(Xa)), pos=0)
    def step(lr):
        if state["pos"] + batch > len(Xa):
            state["o"], state["pos"] = rng.permutation(len(Xa)), 0
        b = state["o"][state["pos"]:state["pos"] + batch]; state["pos"] += batch
        A = d126.forward(net, Xa[b], acts); loss, d = bce_logits(A[-1], ya[b])
        opt.lr = lr; opt.step(net, d126.backward(net, A, acts, d)[0])
        return loss
    return step
```

- `state = dict(o=..., pos=0)` — a dictionary the inner function can change; reassigning a plain name inside it would need nonlocal.
- `def step(lr):` — a function defined inside a function.
- `state["o"], state["pos"] = rng.permutation(len(Xa)), 0` — at the end of the rows, a new order and back to the start.
- `opt.lr = lr` — changes the rate of the existing optimiser, so Adam keeps its moment estimates between steps.
- `return step` — a closure: step remembers net, opt, rng and state after mlp_stepper has returned.

SYSTEM DIAGRAM



One finder run: the rate rises every step; one tenth of the smoothed minimum is suggested.

Segment 2 — The experiments

demo_known: least squares with a known limit

MATH

$$w \leftarrow w - \eta\, X^T(Xw - y)/n \quad \text{diverges for } \eta > 2/\lambda_{max}(X^TX/n)$$

CODE

```
X, y = D["Xa"].astype(float), D["ya"].astype(float)
lam = np.linalg.eigvalsh(X.T @ X / len(X)).max(); crit = 2 / lam
print(f"  largest eigenvalue of X^T X / n: {lam:.4f}; gradient descent diverges above 2 /
    {lam:.4f} = {crit:.4f}")
loss = lambda w: 0.5 * np.mean((X @ w - y) ** 2)
for f in (0.9, 0.99, 1.01, 1.1):
    w = np.zeros(X.shape[1])
    with np.errstate(all="ignore"):
        for _ in range(2000):
            w -= f * crit * X.T @ (X @ w - y) / len(X)
        print(f"  constant rate {f:4.2f} x the limit, 2000 steps: loss {loss(w):.4g}")
w = np.zeros(X.shape[1]); out = {}
def step(lr):
    nonlocal w
    l = loss(w); w = w - lr * X.T @ (X @ w - y) / len(X); return l
for beta in (0.0, 0.98):
    w = np.zeros(X.shape[1])
    with np.errstate(all="ignore"):
        lrs, raw, sm = lr_find(step, lr_range(crit / 1000, crit * 100, 200), beta=beta)
```

- `np.linalg.eigvalsh(X.T @ X / len(X)).max()` — eigvalsh is for symmetric matrices: faster, and its eigenvalues are real.
- `loss = lambda w: ...` — a one-line function stored in a name.
- `with np.errstate(all="ignore"):` — silences numpy's overflow warnings inside the block; the diverged loss itself shows the overflow.
- `nonlocal w` — lets step reassign w, a name of the enclosing demo_known.
- `l = loss(w); w = w - ...; return l` — the loss before the step, as a finder records it.

demo_finder: five seeds, two optimisers

CODE

```
for name, oc in (("Adam", d126.Adam), ("SGD", SGD)):
    F[name] = []
    for seed in range(SEEDS):
        t0 = time.time()
        with np.errstate(all="ignore"):
            lrs, raw, sm = lr_find(mlp_stepper(D["Xa"], D["ya"], oc, seed), lr_range(LO, HI,
                FIND_STEPS))
        s = suggest(lrs, sm); F[name].append(dict(lrs=lrs, raw=raw, sm=sm, sec=time.time() - t0,
            **s))
        print(f"  {name:4s} seed {seed}: {len(sm)} steps, stopped at {lrs[-1]:.2e}; lowest
            smoothed loss {sm.min():.4f} at "
              f"{lrs[np.argmin(sm)]:.2e}; min / 10 suggests {s['min_over_10']:.2e}; steepest
                  {s['steepest']:.2e}")
    for k in ("min_over_10", "steepest"):
        v = np.array([f[k] for f in F[name]])
        print(f"  {name:4s} {k:11s}: suggestions span a factor of {v.max() / v.min():.1f} over
            {SEEDS} seeds")
```

- `(("Adam", d126.Adam), ("SGD", SGD))` — pairs of a name and a class, unpacked by the loop.
- `dict(lrs=lrs, ..., **s)` — ** spreads the suggestions dictionary into the new one.

- `{name:4s}` — a string padded to 4 characters, so the columns line up.
- `v.max() / v.min()` — how far apart the five suggestions are, as a factor.

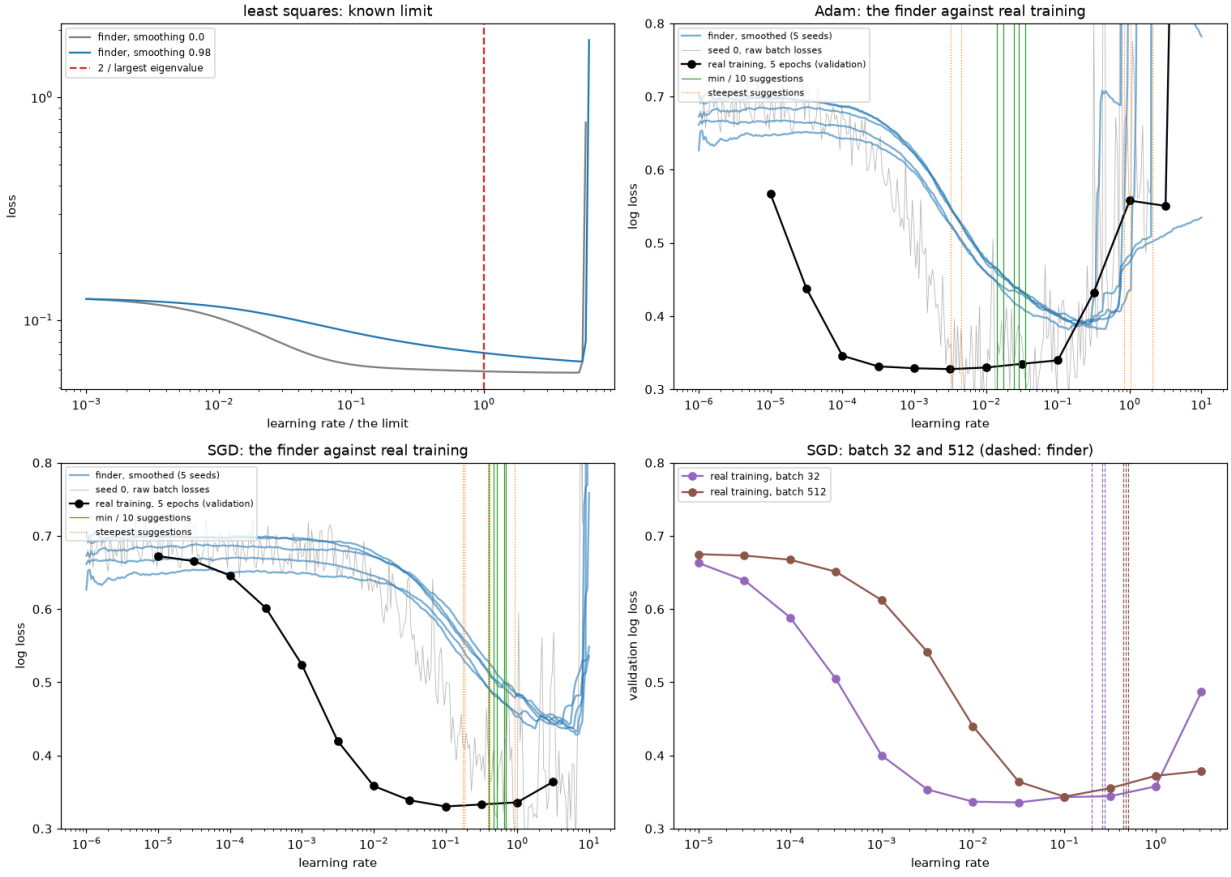
demo_check and demo_batch: real training

CODE

```
    S[name] = np.array([[train_mlp(D["Xa"], D["ya"], D["Xv"], D["yv"], oc, lr, s) for s in
        range(SWEEP_SEEDS)] for lr in GRID])
    m = S[name].mean(1); b = int(np.argmin(m))
    print(f"  {name}: " + ", ".join(f"{lr:.0e} {v:.4f}" if np.isfinite(v) else f"{lr:.0e}
        diverged" for lr, v in zip(GRID, m)))
    good = GRID[m <= m[b] + 0.005]
    f = [suggest(*lr_find(mlp_stepper(D["Xa"], D["ya"], SGD, s, batch=bs), lr_range(LO, HI,
        FIND_STEPS))[::2])["min_over_10"]
         for s in range(3)]
```

- `[[train_mlp(...) for s in ...] for lr in GRID]` — a nested comprehension: 13 rows of 3 seeds.
- `S[name].mean(1)` — the mean over seeds, along axis 1.
- `GRID[m <= m[b] + 0.005]` — a boolean mask keeps the rates within 0.005 of the best.
- `suggest(*lr_find(...)[::2])` — lr_find returns (rates, raw, smoothed); [::2] keeps items 0 and 2, and * passes them as two arguments.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.gradient(y, x)` — the slope of y against x at every point.
- `np.logaddexp(a, b)` — $\log(e^a + e^b)$ without overflow.
- `np.linalg.eigvalsh(A)` — eigenvalues of a symmetric matrix, ascending.
- `np.errstate(all="ignore")` — a with block that silences floating-point warnings.
- `nonlocal name` — reassign a name of the enclosing function.
- `closure` — an inner function that keeps the outer function's names alive.
- `W -= x` — in-place update: changes the array every reference sees.
- `t[::2]` — every second item of a sequence.

End of Day 170 syntax reference. For the experiments and what they showed, see the Day 170 complete guide.